SDQ Technical Doc

March 31, 2020

Prepared by

Robert Saliba

Table of contents

- Prepared by

Table of contents

Solution Overview
- The dashboard
- The Android mobile app

Setup the solution locally

Code structure
- HTML Pages Setup:
- Routing
- APIs used for the app
- Controllers:
- List of APIs consumed by the app

Setup the Android app locally

Generate a new APK

Application Flow

Import the database

Code auto generation logic and source code
- What happens in the background

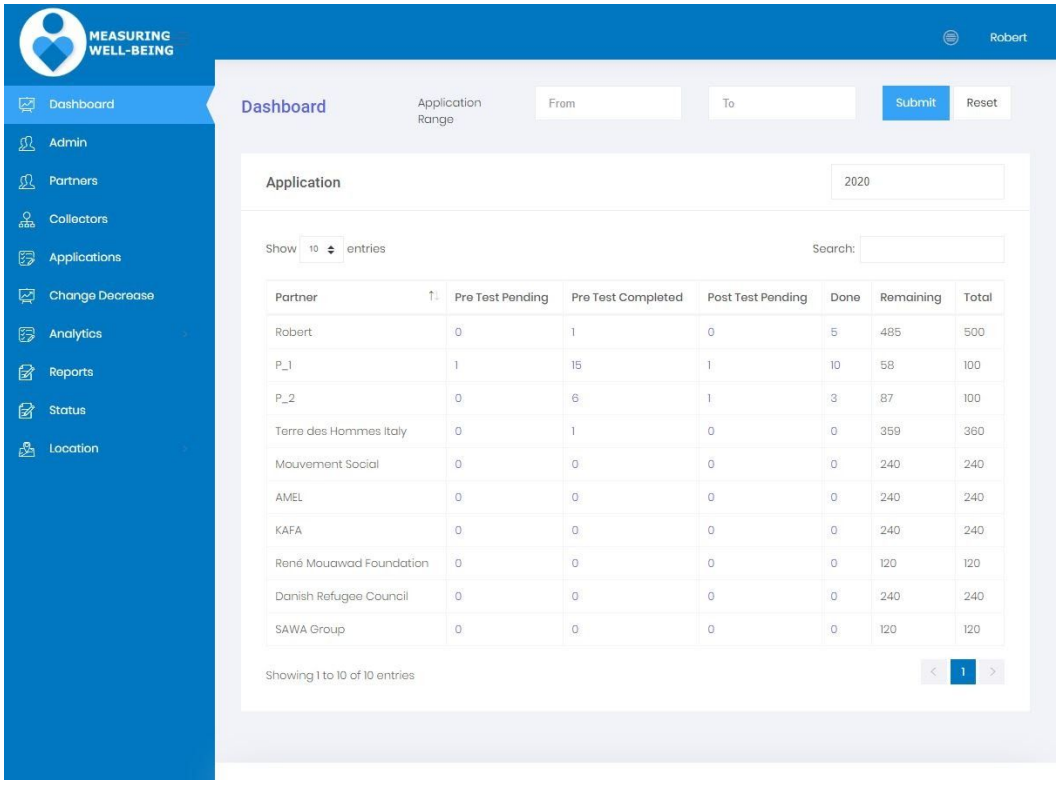
Solution Overview

The solution consist of two main components:

The dashboard

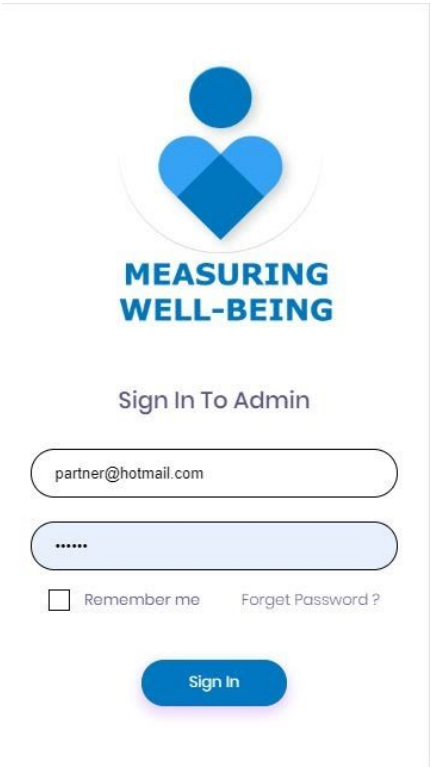
Built using Laravel 5.8

Hosted on Godaddy



More details about how to use the dashboard in the user guidelines document attached.

The Android mobile app



The App APK file is located in the folder below

https://prod.measuringwellbeing.org/apk/sdqx.apk

X in the name should include the version name

So far we are in version 5 at the moment this document is written so the link is

https://prod.measuringwellbeing.org/apk/sdq5.apk

Setup the solution locally

We are using Laravel version 5.8 .

Laravel Documentation Link: https://laravel.com/docs/5.8/installation

1. Download and Install git
    a. https://git-scm.com/downloads
2. Download and Install wamp or xamp on your machine
    a. http://www.wampserver.com/en/#download-wrapper
3. Run wamp and make sure the color of the icon is green
4. Inside wamp Go inside the www folder, usually located under
    a. C:\wamp64\www\
5. Create folder and call it for example **sdq**
6. Right click on **sdq** and click on "**git bash here**"
7. Clone the git url in your local system.
    a. Git clone sdq .
8. Or copy the zipped source code inside the www folder
9. Click on wamp icon on
10. Create a blank database in phpmyadmin and import the database file from database -> Backup -> sdq.sql or attached database file
11. Go to the root of the solution in www/sdq/ and rename the ".env.example" file to ".env" to make configurations.
12. Open the .env file and change the database settings as per your local system.
    a. Set DB_DATABASE, DB_USERNAME, DB_PASSWORD for database settings.
    b. [Production only and not local] Set MAIL_HOST, MAIL_PORT, MAIL_USERNAME, MAIL_PASSWORD, MAIL_ENCRYPTION variable values to make emails working.
13. After performing the above steps run the below commands.
    a. composer install / composer update
    b. php artisan cache:clear
    c. php artisan view:clear
    d. php artisan route:clear
    e. php artisan cache:clear

f. php artisan config:cache

14. After this you will be able to access the site in your browser.
    a. http://localhost/sdq

Code structure

1. HTML Pages Setup:

a. Go to the root folder of the project -> resources -> view here we can define the pages.
b. In the “view” folder we are having “admin” folder which contains all the pages for backend panel
c. Each module has it’s individual folder that contains the pages for it.
d. “layout” folder under the “admin” folder contains the main files of the partials.
e. Inside the “layout” folder we have app.blade.php file which is the main file which loads the main HTML tag and assets like CSS, Javascripts, etc.
f. Inside the “layout” folder having a file sidebar.blade.php which loads the sidebar of the backend panel.
g. Inside the “layout” folder having a file header.blade.php which loads the header of the backend panel.
h. Inside the “layout” folder having a file footer.blade.php which loads the footer of the backend panel.
i. View files must have “.blade.php” type. Blade is a file type of laravel which is used in laravel.

2. Routing

a. In the root folder we will have routes -> web.php file it will contain all the routes and url of the site. These endpoints are called by the internal controllers of the solution. Each and every url must be defined here otherwise it will not work. Example: Route::get('/login','Auth\LoginController@index')->name('login');

3. APIs used for the app

a. API endpoints are used for the Android app to connect, get data and save data to our dashboard

b. The endpoint should be pointing to this url

i. https://measuringwellbeing.org/prod/api/

4. **Controllers:**
    a. To create a controller run this command:
        i. php artisan make:controller NameController
    b. Controllers will be in this path => app -> Http -> Controllers
5. **List of APIs consumed by the app**
    a. https://measuringwellbeing.org/prod/api/login for the login
        i. The solution should return a hashed token and stored in the database that needs to be used with all the other calls to identify the user
    b. https://measuringwellbeing.org/prod/api/get-application?token=xxx
        i. This API return all applications for the collector
    c. https://measuringwellbeing.org/staging/api/submit-pre-test-application?token=xxx
        i. Submits a pretest
    d. https://measuringwellbeing.org/staging/api/submit-post-test-application?token=xxx
        i. Submits a postest

Setup the Android app locally

1. Download Android Studio from
    a. https://developer.android.com/studio
    b. I am using version 3.5
2. Click on file -> Open -> select the folder i sent you in the .zip file **androidapp.zip**
3. Let the solution build and download dependencies
4. It might take up to 20-30 min to finish depending on the internet speed.
5. After it is finished you will be good to go and you'll get green checkboxes at each dependency in the console at the bottom part of the screen.
6. On the left bar, you have the "Project" tab

7. The code is inside **app -> java folder**

Generate a new APK

1. Click on BUILD from the top menu bar.
2. Hover to Build bundle(s) or APK and Select Build APK.
3. After Build is Complete a Popup will appear from the right bottom.
4. Click LOCATE.
5. Please upload it in the apk folder inside https://prod.measuringwellbeing.org/apk/sdqX.apk and respect the naming convention sdq(version).apk

Application Flow



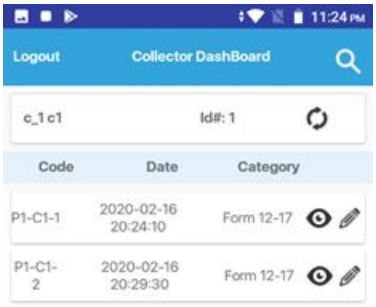
Import the database

1. Go to phpmyadmin and create a database, call it sdq
2. Import the sdq.sql from the import section, upload the database
3. Usually the local phpmyadmin username is root and password is empty
4. Make sure the database information is the same as you have in the .env file you have in the solution we defined before

Code auto generation logic and source code

The system will be auto generating codes for each application that is included in the sample population. It is generated automatically by the SDQ app or the web version, no more manual creation, each partner will have a specific set of applications per year.

Each Partner will have a Prefix during the addition to the system, same for the collector.



Example for Partner **Robert** the prefix could be **ROB** and collector **Michel** working for Partner Robert it will be for example **MICH**.

So the application auto generated code will be **Partner Prefix + Collector Prefix + application ID** in this case for example It will be **ROB-MICH-1**.

In Online and offline mode, the application id is auto increment by 1, so the next saved or submitted application will be **ROB-MICH-2, ROB-MICH-3** …..

The code will be shown in the app after the collector submits the application in the post-pending page.



In case we **Save** an application without submit, it will be at the bottom of the respective application status. Example: if I am in a pre-test application and I save it, the application should show in the pre-test pending screen. If I am in the post-test application and save the application/application id should show in the post-test pending screen, the last one.

What happens in the background

While hitting the login api for collector authentication, the server return a set of keys that the application will catch and try to auto generate the code ex:

Calling for **/api/login** of type **POST**

Body type raw contains this payload

```
{
"email":"collector1@akkar.akkar",
 "password":"123456"
}
```

Will return on success this payload below, highlighted in green the keys used


```
{
   "success": true,
   "data": {
      "id": 44,
      "firstname": "collector1",
      "lastname": "collector1",
      "email": "collector1@akkar.akkar",
      "phone": "5146494048",
      "zone": null,
      "address": "2250 guy\r\napp 807",
      "fax": null,
      "is_active": "1",
      "email_verified_at": null,
      "is_readonly": "0",
      "api_token": "LYrw22JRzwkTBaVYz8OzJjqJamzb5OejZD1hc13GFUlODPw7pA",
      "created_at": "2020-03-17 19:32:08",
      "updated_at": "2020-04-02 01:07:59",
      "collector_prefix": "C1",
      "partner_prefix": "AKK",
      "partner_name": "Akkar",
      "last_increment_no": 1
   },
```

"message": "Welcome! Collector1"
}

Warning:

A collector can only use the Android application or the Web Version to submit applications not both. The reason is because of the offline application saved on the collector device if any. So the edge case is the following, in offline mode if a user submits an application with no internet connection, the application will be stored on the collector device, in the SQLite database and when he's connected to the internet the applications will be synched of course with each application with the auto generated code. The server in this case doesn't know that the collector added new applications with new codes. The collector goes back to his Web version and submits applications using the web version, if his phone goes back online at that stage or after we will be having issues with the codes.

Race conditions are hard to catch nearly impossible to fix especially if working in offline mode so I recommend using best practice in case the collector decides to use both, first is to make sure when he goes back online to sync the offline applications he has on his phone before resuming working on the web version. He can double check his new applications are shown in the collector online dashboard.

The application code is being generated on the application level and not the server level.

FTP Access to the production server

FTP Username: achamseddine@measuringwellbeing.org

FTP Password:?u&RipF0&XHy

FTP server: measuringwellbeing.org

FTP & explicit FTPS port: keep it empty

Cron jobs happening on the website

Daily backup for Database and source code

Anti malware and Anti virus